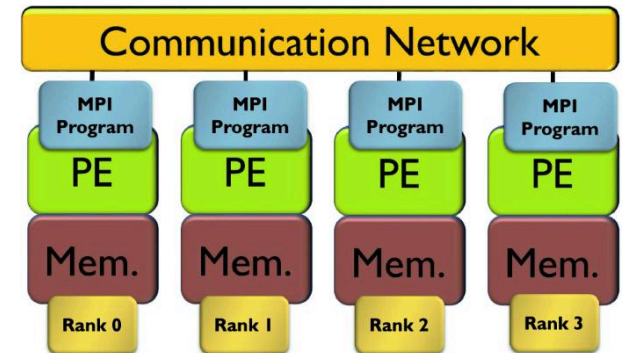
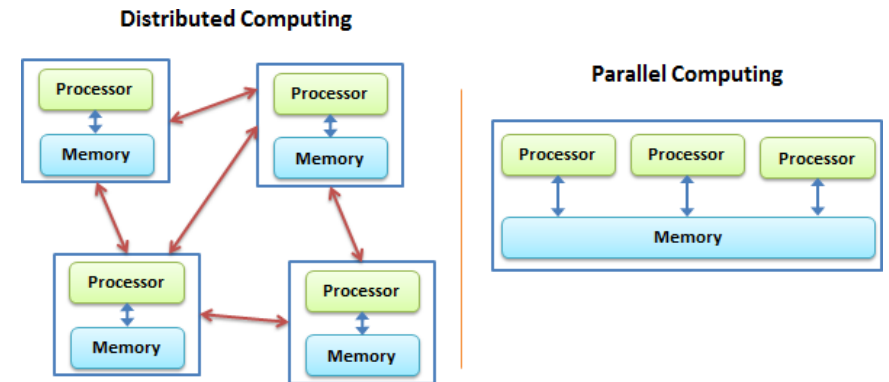
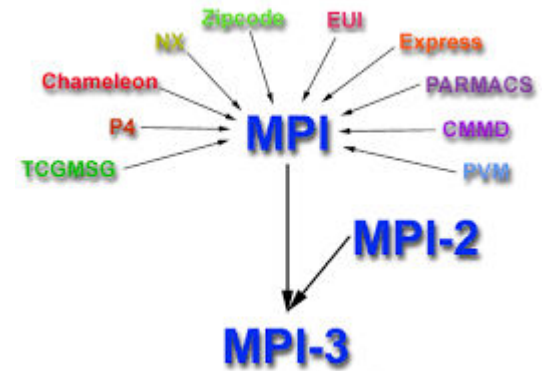


MPI 병렬 프로그래밍

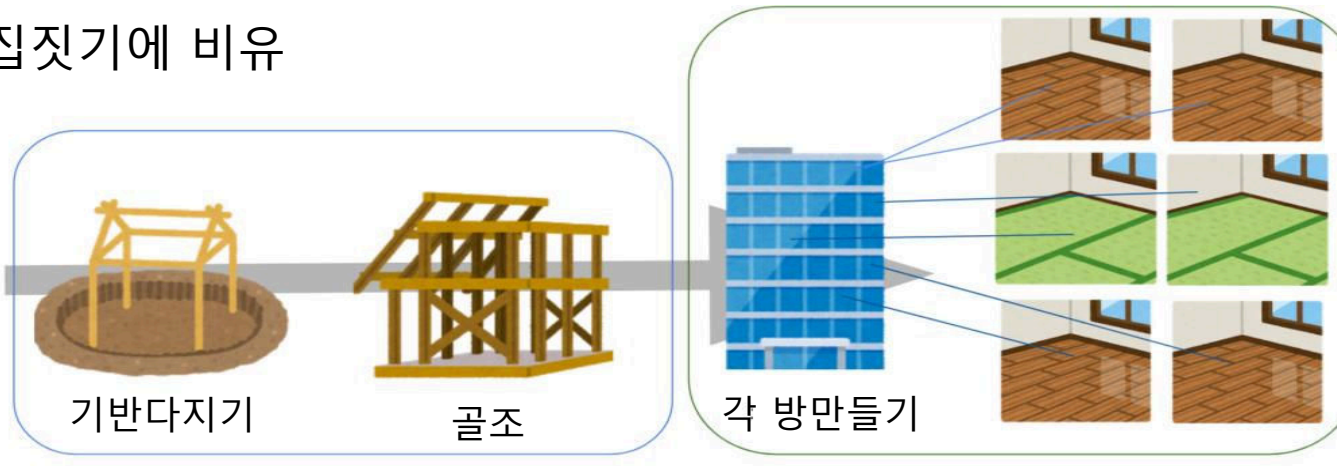


MPI를 통한 병렬 컴퓨팅 프로그래밍

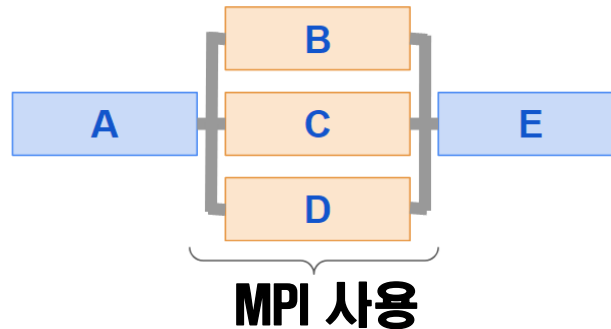
병렬 계산

- 기본적으로 병렬로 계산하기 쉬운 부분만 병렬로 계산.
- 집짓기에 비유하자면, 기초는 순차적으로 계산해야 하지만 각 방은 병렬로 진행할 수 있는 식.

집짓기에 비유



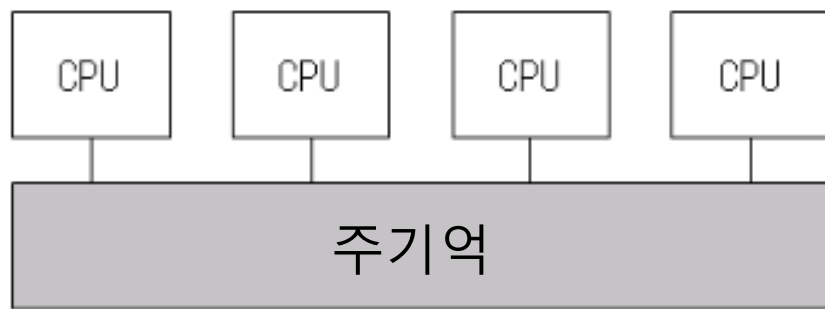
- 프로그램으로 생각해보면, 프로그램 내 for 루프 부분처럼 계산 시간이 많이 소요되는 부분만 병렬로 계산.



병렬 처리 모델

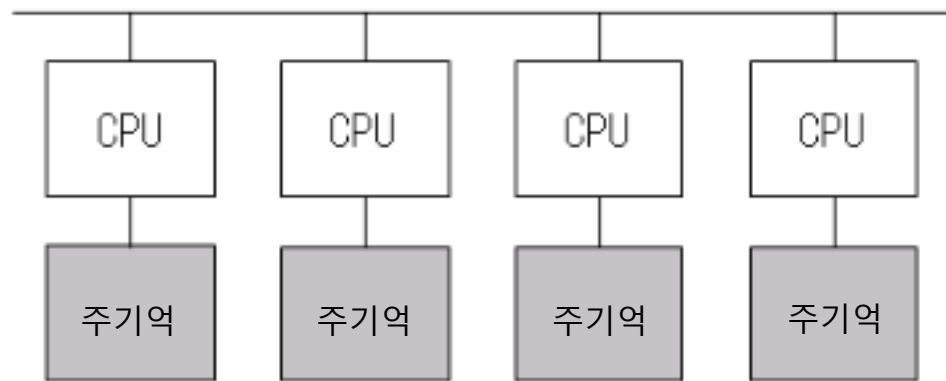
- 컴퓨터 아키텍처에 따른 처리 분담(분할) 방식에 따라 몇 가지 병렬 처리가 있다.

공유메모리 병렬처리



분산메모리 병렬처리

MPI(Message Passing Interface)는 분산 메모리 병렬 처리를 위한 병렬 기법이다.



MPI를 통한 병렬 컴퓨팅 프로그래밍

- MPI는 Message Passing Interface라고 불리는 병렬 연산 표준.
- 서로 다른 머신(노드) 간에 계산을 할 때 이용되는 기능이기 때문에 서버의 대규모 계산에는 MPI가 이용됨
- 2005년경부터 싱글코어의 성능이 포화상태에 이른 오늘날, 병렬연산은 매우 강력하고 주목도가 높은 기술

MPI와 메모리

- MPI는 분산 메모리라는 구성에 활용됨
- 분산 메모리는 CPU와 메모리가 일대일로 짝을 이룬 상태에서 그 짝이 동일한 네트워크에 있는 상태를 말한다. 이 쌍을 노드라고 합니다.



- [Ubuntu-우분투-에서 openmpi 설치하기 \(tistory.com\)](https://tistory.com/entry/ubuntu-ubuntu-에서-openmpi-설치하기)

```
sudo apt install mpich
```

MS-MPI인스톨

- <https://www.microsoft.com/en-us/download/details.aspx?id=100593>

Important! Selecting a language below will dynamically change the complete page content to that language.

Select language

English ▾

Download

Choose the download you want

☐ File Name	Size
☐ msmsetup.exe	7.5 MB
☐ msmisdk.msi	2.2 MB

Download

Total size: 0 bytes

▼ Install Instructions

To install on a workstation computer:

1. Run MSMpiSetup.exe or msmisdk.msi..
2. Follow the steps in the installation wizard.

For redistribution:

- Please refer to the license agreement for additional information.

▼ System Requirements

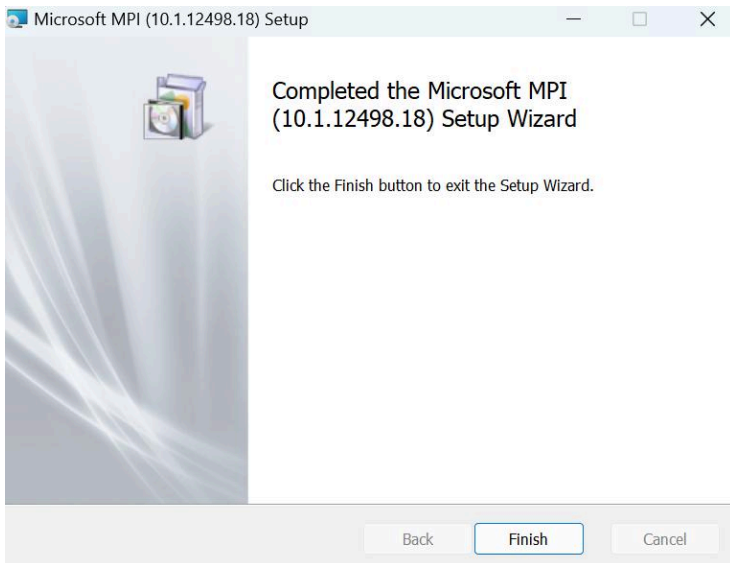
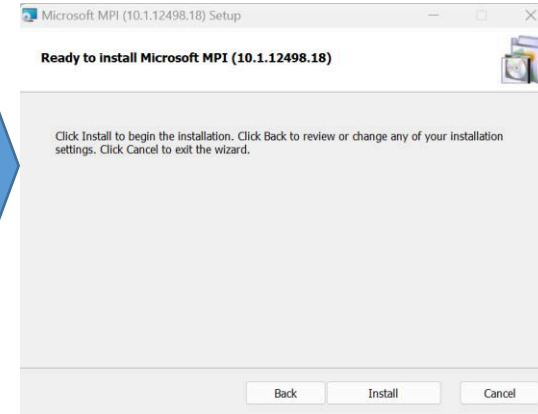
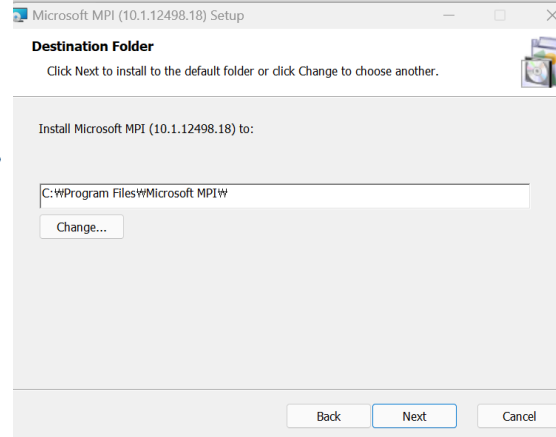
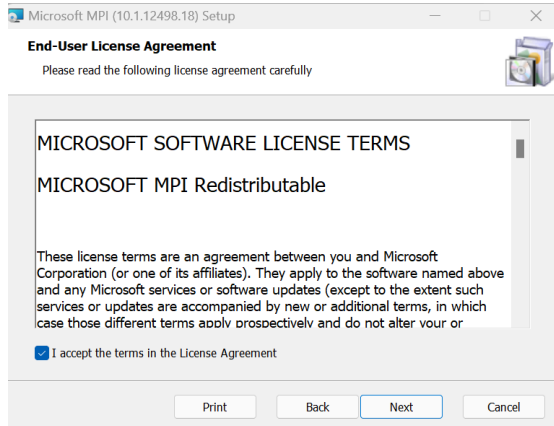
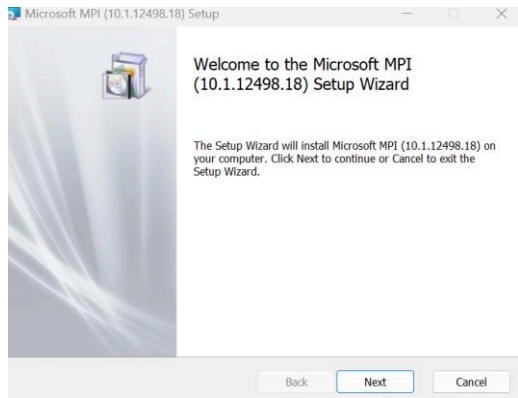
Supported Operating Systems

Windows 7, Windows 8.1, Windows 10, Windows Server 2012 R2, Windows Server 2016, Windows Server 2019

Windows 10, Windows 8.1, Windows 8, Windows 7, Windows Server 2019, Windows Server 2016, Windows Server 2012 R2, Windows Server 2012, Windows Server 2008 R2

MPI를 통한 병렬 컴퓨팅 프로그래밍

MS-MPI인스톨



- 분산 메모리형 병렬 컴퓨터에서의 병렬 프로그래밍을 위한 프로세서 간 통신 라이브러리
- 기본 언어(FORTRAN/C/C++)에 프로세서 간 통신을 하는 서브 루틴/함수를 추가하여 병렬 프로그래밍을 할 수 있도록 확장
- 1992년경부터 미국 컴퓨터 제조업체 및 대학을 중심으로 표준화
- MPI의 규격
 - 1995 MPI: Message Passing Interface Standard
 - 1997 MPI2: Extensions to the Message Passing Interface

분산 메모리형 병렬 계산기(복습)

- 아키텍처
 - 각기 메모리를 가진 여러 개의 PU를 네트워크로 연결
 - 각 PU는 각각 자신이 가진 메모리에만 접근 가능
- 특징
 - 수천~수만 PU 규모의 병렬 처리 가능
 - PU 간 데이터 분산을 의식한 프로그래밍 필요
- 프로그래밍 언어
 - FORTRAN/C/C++ + MPI 사용

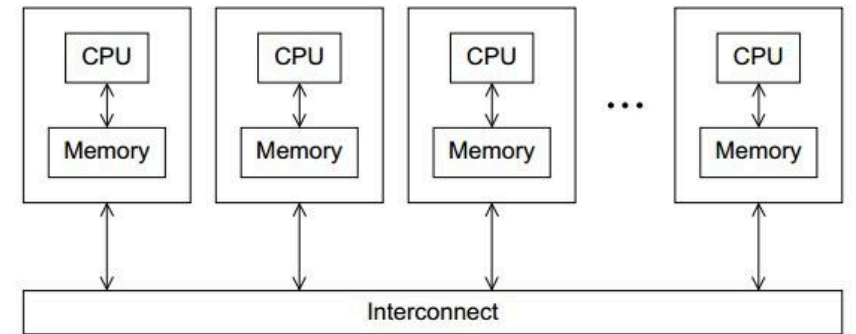


FIGURE 3.1

A distributed-memory system

MPI에 의한 통신: 우편물의 우편과 같음.

- 우편 발송에 필요한 정보:

1. 자신의 주소, 발송지 주소
2. 안에 들어있는 물건이 어디에 있는지
3. 안에 들어있는 물건의 분류
4. 안에 들어있는 물건의 양
5. (여러 개의 짐을 동시에 보내는 경우) 인식 방법
(태그)

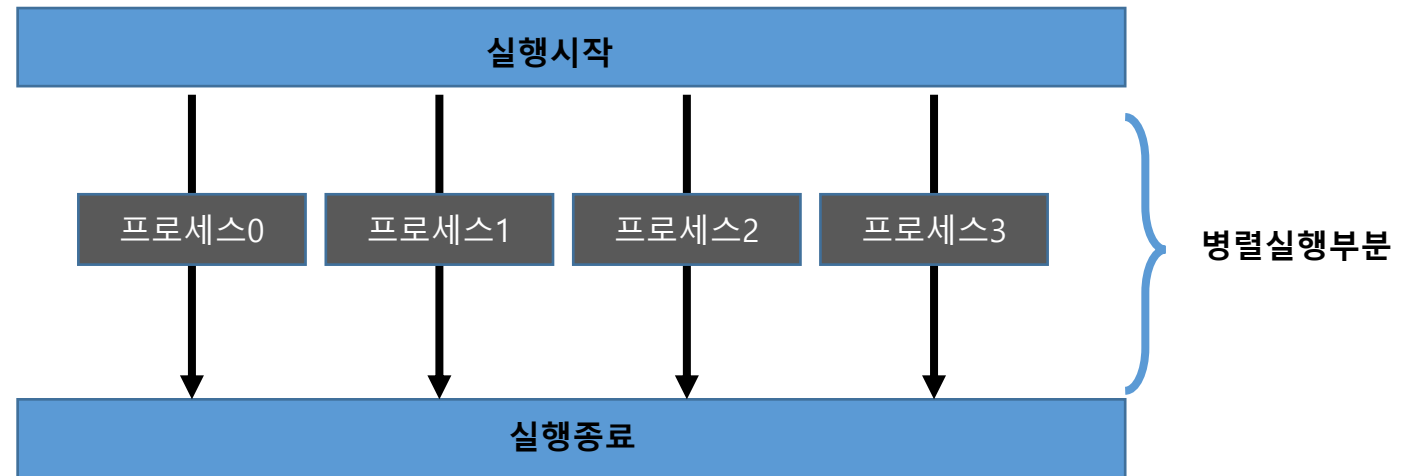
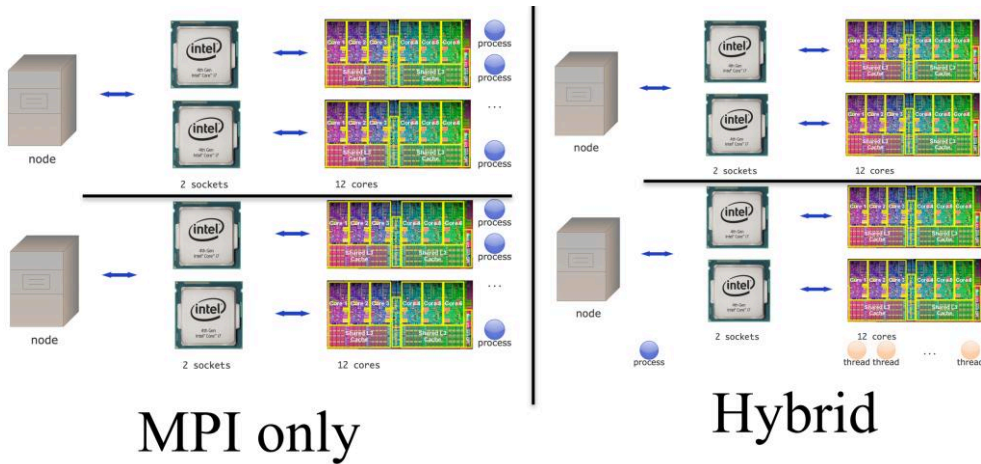


- MPI에서는

1. 본인 및 발송인의 인식 ID
2. 데이터 저장위치의 주소
3. 데이터 유형
4. 데이터 양
5. 태그 번호

■ 실행 모델

- 실행 시작부터 종료까지 모든 프로세스가 동일한 프로그램을 실행한다. (SIMD)
- 각 프로세스는 고유한 프로세스 번호(rank)를 가진다.
- P 개의 프로세스로 실행할 때, 랭크는 0에서 P-1까지의 정수
- 연산 대상 데이터나 IF 문에 의한 분기 등은 프로세스마다 다를 수 있다.



OS/Software

Process	Independent stream of instructions. OS provides dedicated resources
Thread(s)	Created by process Shares resources with process/fellow threads

Hardware

core	“instruction stream processing unit”
“Processor/CPU”	Single die that sits on a “Socket” (has 1 or more cores)
Node	~ “mother board”. With 1 or more sockets
Blade	1 or more Nodes
Cabinet	Several blades

■ 메모리 공간

- 프로그램 내에서 변수/배열을 정의하면, 같은 이름의 변수/배열이 프로세스마다 독립적으로 할당된다.
 - ✓ 같은 이름의 변수/배열이라도 프로세스마다 다른 값을 가질 수 있다.
- 다른 프로세스가 가지고 있는 변수/배열에 접근할 수 없다.

```
int a;  
int b[100];
```

변수 a, 배열 b[100]가 각 프로세스에 대해 각각 독립적으로 할당된다.

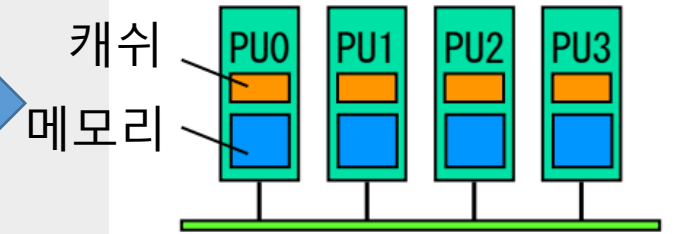
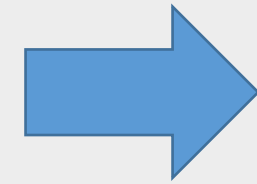
■ 프로세서 간 통신

- 다른 프로세스가 가지고 있는 변수/배열의 값을 참조하기 위해서는 프로세서 간 통신을 통해 해당 변수/배열의 값을 전송받아야 한다..
- 이를 위한 통신 함수 집합이 MPI

MPI 실행 환경

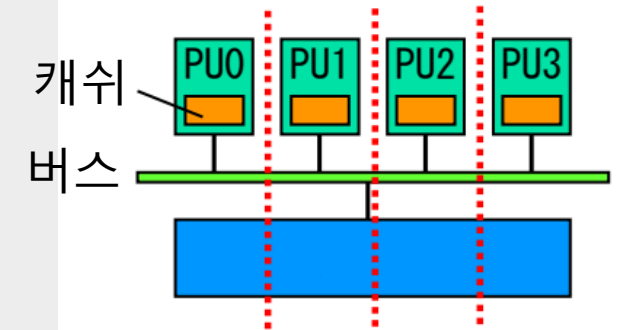
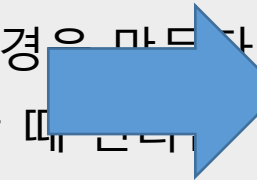
■ 분산 메모리형 병렬 계산기

- MPI가 원래 가정하고 있는 환경
- 각 프로세스가 1개의 PU에 할당되는 환경



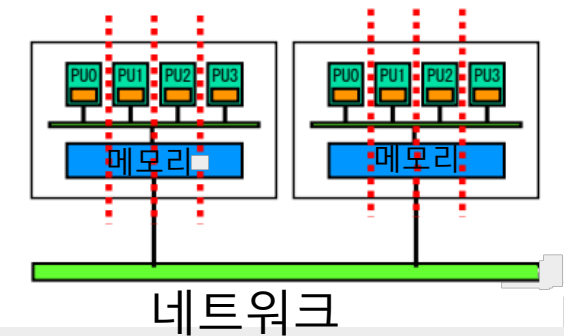
■ 공유 메모리형 병렬 컴퓨터

- 메모리를 분할하여 각 PU에 할당하여 가상의 분산 메모리 환경을 만든다.
- 기존 MPI 프로그램을 공유 메모리형 병렬 컴퓨터에서 실행할 때 쓴다.
- 본 강의에서도 이 환경을 사용한다.



■ SMP(Symmetric multiprocessing) 클러스터

- 각 노드의 메모리를 분할하여 각 PU에 할당하여 가상의 분산 메모리 환경을



MPI 프로그램의 구성요소

■ 원본 프로그램

- 일반적인 FORTRAN 또는 C/C++로 작성된 프로그램
- 분산 메모리를 의식하여 작성되어야 한다.

■ 환경 설정 및 환경 획득을 위한 함수

- MPI의 초기화, 종료 처리를 하는 함수
- 자기 프로세스의 Rank(고유식별자), 전체 프로세스 수를 취득하는 함수

■ 통신 함수

- 프로세스 간 통신을 위한 서브 루틴/함수
- 일대일 통신과 집단 통신이 있다.

■ 기본 데이터 타입과 미리 정의된 연산

- MPI_INT, MPI_FLOAT, MPI_DOUBLE, MPI_CHAR 등
- 통신 함수의 데이터 타입 지정 및 통신과 연산을 동시에 수행하는 함수(합산 등)의 연산 지정에 이용한다.

■ 인클루드 파일

- 프로그램 시작 부분에 `#include <mpi.h>`라고 기술한다.

■ 실행 시 지정

- mpirun 명령어로 프로그램 실행에 사용할 프로세스 수를 지정한다.

MPI 프로그램의 구성요소

대표적인 MPI 함수

- 시스템 함수
 - MPI_Init(), MPI_Comm_rank(), MPI_Comm_size(), MPI_Finalize()
- 일대일 통신 함수
 - 블로킹형: MPI_Send(), MPI_Recv()
 - 비차단형: MPI_Isend(), MPI_Irecv()
- 1 대 전체 통신 함수
 - MPI_Bcast()
- 집단 통신 함수
 - MPI_Reduce(), MPI_Allreduce(), MPI_Barrier()
- 시간 측정 함수
 - MPI_Wtime()

■ 이식성

- MPI로 프로그램을 작성하여 다양한 분산 메모리형 병렬 컴퓨터에서 실행 가능
- 공유 메모리형 병렬 컴퓨터에서도 실행 가능

■ 확장성

- 적절한 알고리즘에 따라 병렬 프로그램을 작성하면 수천~수만 개의 프로세서를 가진 병렬 컴퓨터의 능력을 끌어낼 수 있다.

■ 다양한 집단 통신

- 합산, 브로드캐스트 등 다양한 통신 기능이 미리 준비되어 있다.

■ 성능 분석 도구

- 부하 분산, 통신 시간 등 상세한 정보를 얻을 수 있는 툴 사용 가능

현재 사용 가능한 MPI 구현

■ MPICH

- 미국 Argonne National Laboratory에서 제공하는 무료 MPI
- 현재 MPICH2가 주류
- <http://www.mpich.org>

■ OpenMPI

- MPI에 관한 다양한 선행 프로젝트들의 성과를 바탕으로 개발된 MPI
- <http://www.open-mpi.org/>

■ 각 컴퓨터 제조업체에서 제공하는 MPI

- 각 병렬 컴퓨터 제조사들이 자사 컴퓨터용 MPI 구현을 제공하고 있음 구현을 제공하고 있다.

프로그램 설명

■ MPI_Init 및 MPI_Finalize

- 프로그램의 시작과 끝에서 호출하여 각각 MPI 초기화, 종료 처리를 수행한다.

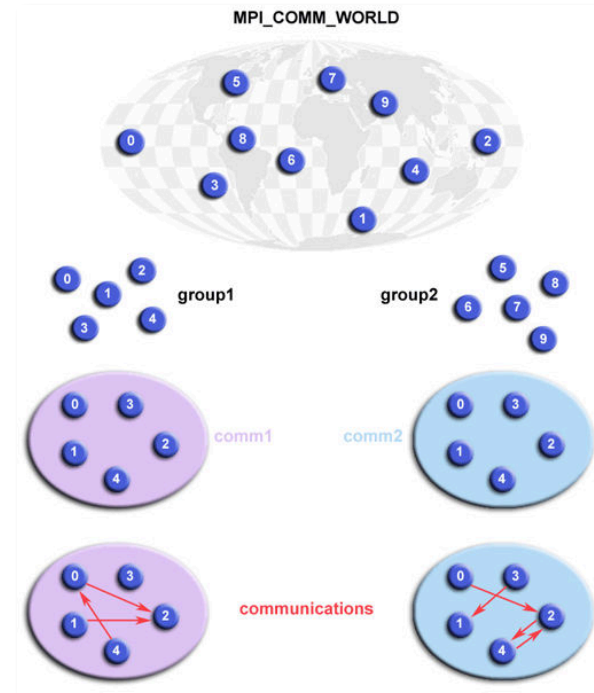
■ int MPI_Comm_size(MPI_Comm comm, int *nprocs)

- comm은 프로세스 그룹을 지정하는 커뮤니케이터
 - ✓ MPI_COMM_WORLD로 하면 모든 프로세스를 포함하는 그룹을 의미한다.
- comm 내의 모든 프로세스 수가 nprocs에 들어간다.
- 성공하면 MPI_SUCCESS가 함수 값으로 반환된다.

■ int MPI_Comm_rank(MPI_Comm comm, int *myrank)

- comm 내의 자신의 프로세스 번호가 myrank에 들어간다.

이 함수들은 어떤 MPI 프로그램에서도 샘플 프로그램과 같은 순서로 호출하면 된다.



MPI 개요

- 분산 메모리 병렬 처리에서 메시지 패싱(Message Passing Interface)의 표준 규격
 - ✓ 여러 프로세스 간 데이터 교환을 위해 사용하는 메시지 통신 연산에 대한 사양 표준
- FORTRAN, C에서 호출하는 서브 프로그램 라이브러리
- 이식성이 우수함
 - ✓ 표준화된 라이브러리 인터페이스를 통해 다양한 MPI 구현 환경에서 동일한 소스를 컴파일하고 실행할 수 있다.
- 프로그래머의 부담이 상대적으로 적다.
 - ✓ 프로그램을 분석하여 데이터-처리를 분할하고, 통신의 내용과 타이밍을 사용자가 직접 작성해야 한다.
- 대용량의 메모리 공간 이용 가능
 - ✓ 여러 노드를 이용하는 프로그램 실행으로 큰 메모리 공간을 사용할 수 있다.

MPI

- 자주 사용되는 MPI 소프트웨어는 다음과 같다.
 - MPICH : 메이저
 - OpenMPI : 메이저
 - MVAPICH : GPU에 강함
- 위 두 가지가 가장 대표적인 MPI 소프트웨어(라이브러리).
- 기본적으로 어느 것을 사용해도 비슷한 결과를 얻을 수 있기 때문에 어느 것을 선택해도 무방
- 예를 들어, MPICH로 MPI 프로그램을 컴파일하려면 mpicc 명령을 사용.

1) mpicc -o mpi_exe mpi_program.c

- 위와 같이 mpi_program.c라는 프로그램에서 mpi_exe라는 실행 파일을 생성할 수 있음.
- C++에서는 ' mpic++ ' 로 컴파일할 수 있음.
- 실행은 mpirun으로 실행.

MPI

2) mpirun -np 8 ./mpi_exe

- 위에서는 np로 병렬 수를 지정하여 8개의 병렬로 계산이 이루어진다

- 노드 지정에는 -host를 사용함.

3) mpirun -host node1,node2 -np 8 -ppn 4 ./mpi_exe

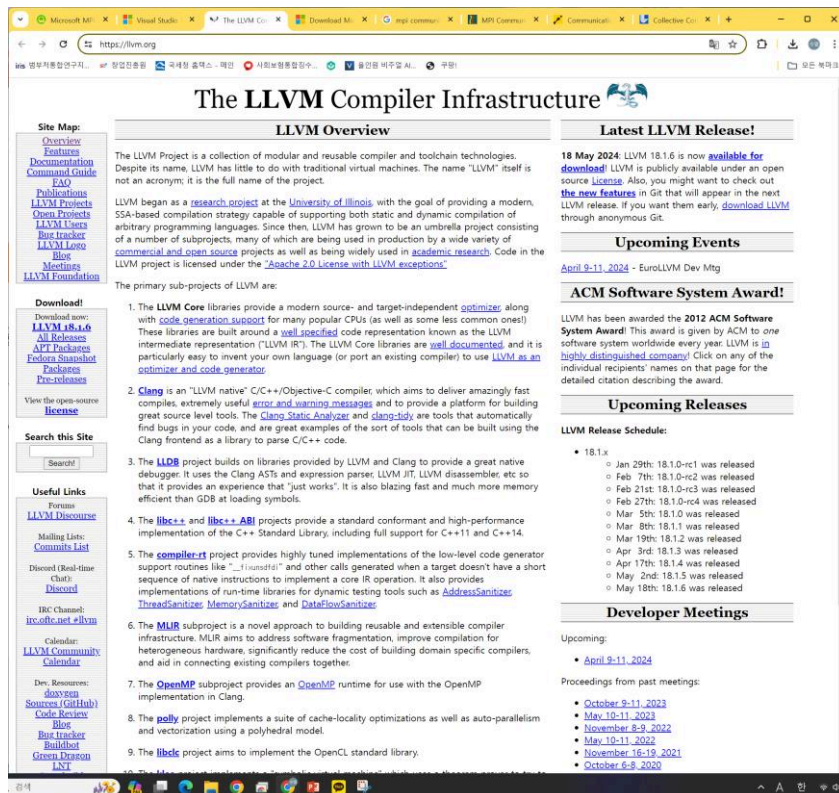
- -ppn에서 각 노드의 병렬 수를 지정.

MPI를 통한 병렬 컴퓨팅 프로그래밍

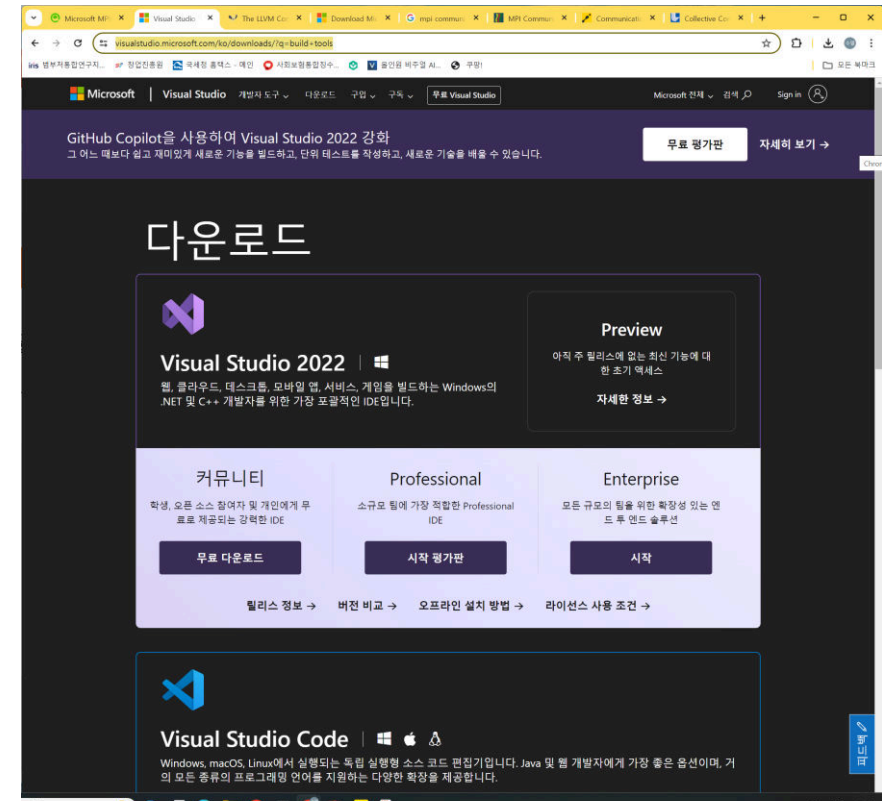
컴파일러

- clang++를 이용하여 명령행으로 컴파일합니다. clang++를 이용하기 위해 llvm을 다운로드하여 설치합니다. llvm과 함께 MSBuild tools를 설치합니다. clang++ 명령어를 명령 프롬프트로 실행하기 위해 llvm/bin 경로를 설정함 `set PATH=%PATH%;"C:\Program Files\LLVM\bin"`

clang++ 설치
<https://llvm.org/>



MSBuild tools 설치
<https://visualstudio.microsoft.com/ko/downloads/?q=build+tools>



```
clang++ sample.cpp -I"C:\Program Files (x86)\Microsoft SDKs\MPI\Include" -L"C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64" -lmsmpi
```

sample.cpp

```
#include <iostream>
#include <mpi.h>
using namespace std;
int main(int argc, char*argv[])
{
    int size, rank;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    cout << "Hello world I am " << rank << " of " << size << endl;
    MPI_Finalize();
    return 0;
}
```

실행

`mpiexec -n 7 a.exe`

- MS-MPI를 설치하면 환경 변수 PATH에 "C:\Program Files\Microsoft MPI\Bin\"이 자동으로 추가되지만, 만약 mpiexec이 없다고 하면 아래와 같이 전체 경로로 실행하거나 환경 변수 PATH에 "C:\Program Files\Microsoft MPI\Bin\"을 추가한다. Program Files\Microsoft MPI\Bin\"을 환경 변수 PATH에 추가한

"C:\Program Files\Microsoft MPI\Bin\mpiexec.exe" -n 7 a.exe

```

Hello world I am 3 of 7
Hello world I am 2 of 7
Hello world I am 6 of 7
Hello world I am 0 of 7
Hello world I am 5 of 7
Hello world I am 1 of 7
Hello world I am 4 of 7
    
```

```

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}
    
```

Any Cluster
\$ mpirun -n 8 ./a.out
SahasraT
\$ aprun -n 8 ./a.out

Hello from 0.
 Hello from 4.
 Hello from 2.
 Hello from 1.
 Hello from 3.
 Hello from 6.
 Hello from 7.
 Hello from 5.

```

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

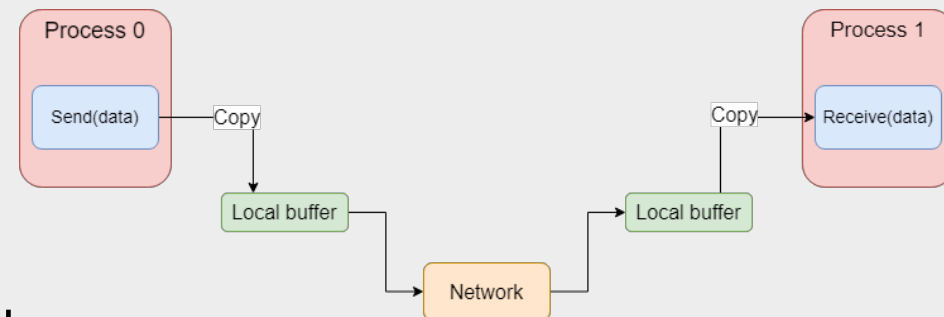
#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}

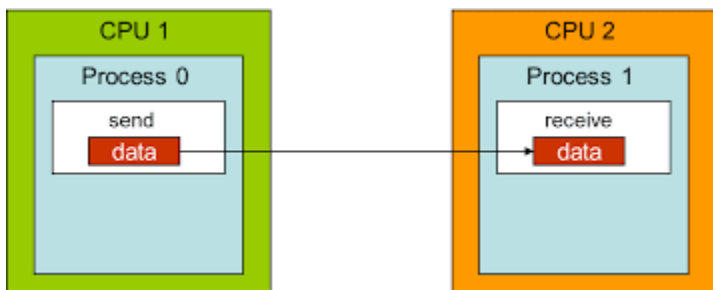
#include "mpi.h"
int main( int argc, char *argv[] ){
    int nproc, myrank;
    MPI_Init(&argc,&argv);
    MPI_Comm_size(MPI_COMM_WORLD,&nproc);
    MPI_Comm_rank(MPI_COMM_WORLD,&myrank);
    printf("Hello from %d.\n",myrank);
    MPI_Finalize();
    return 0;
}
    
```

MPI의 주요 기능

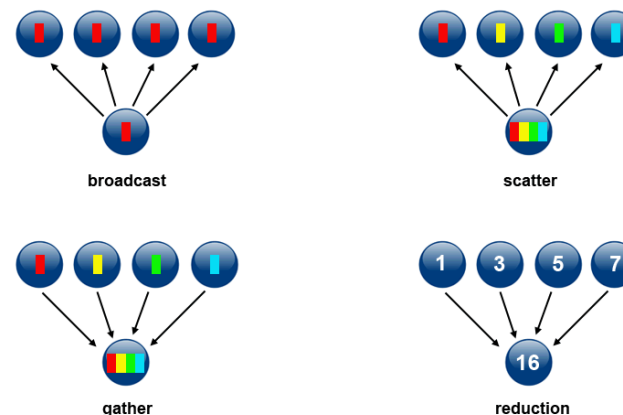
- 프로세스 관리
 - MPI 프로그램 초기화 및 종료 처리 등 수행
- 일대일 통신
 - 일대일로 하는 통신
- 집단 통신
 - 그룹 내 프로세스 전체가 관련된 통신 조작



● 일대일 통신



● 집단 통신



병렬 계산

- 즉, 병렬 연산을 사용한다고 해서 코어 수만큼 속도가 빨라지는 것은 아님.
- 프로그램의 내용, 즉 병렬 부분의 개수에 따라 병렬화 효율이 달라지므로 주의해야 함.
- 이러한 병렬화 용이성을 확장성이라고 함

MPI 프로그램(기본)

<https://rantahar.github.io/introduction-to-mpi/aio/index.html>

- 아래는 샘플 프로그램
- 여기서는 기본이 되는 최소한의 프로그램을 보여주고 있으며, 데이터 교환은 하지 않음.

```
1. #include <stdio.h>
2. #include "mpi.h"
3. int main(int argc, char *argv[]){
4.     int nproc,rank;
5.     MPI_Init(&argc, &argv);
6.     MPI_Comm_size(MPI_COMM_WORLD, &nproc);
7.     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
8.     printf("%d in %d", rank, nproc);
9.     MPI_Finalize();
10.    return 0;
11.}
```

- 먼저 mpi를 사용하려면 mpi 라이브러리를 포함시켜야 함. 그래서 두 번째 줄에서 mpi.h를 포함시키고 있음.
- main 함수 내에서 대문자 MPI로 시작하는 것이 MPI의 API 이고, 이러한 API를 이용하여 다수의 프로세스를 관리.
- MPI를 사용하기 위해서는 5번째 줄의 MPI.Init에 의한 초기화와 9번째 줄의 MPI.Finalize에 의한 종료가 필수. 이 사이에 MPI의 API를 사용할 수 있게 됨.
- 6번째 줄의 MPI_Comm_size는 프로세스 수를 가져옴.
- 첫 번째 인수인 MPI_COMM_WORLD는 MPI 전체를 나타내는 변수이며, 두 번째 인수인 nproc에 프로세스 수를 부여하고 있음.
- 7번째 줄의 MPI_Comm_rank도 비슷한 기능으로, 각 프로세스가 담당하는 자신의 프로세스 번호를 얻을 있음. 이렇게 하면 모든 프로세스가 서로 다른 번호를 얻게 됩니다(ID를 가짐).

MPI 프로그램(기본)

- 먼저 mpi를 사용하려면 mpi 라이브러리를 포함시켜야 함. 그래서 두 번째 줄에서 mpi.h를 포함시키고 있음.
- main 함수 내에서 대문자 MPI로 시작하는 것이 MPI의 API이다.
- 이러한 API를 이용하여 다수의 프로세스를 관리.
- MPI를 사용하기 위해서는 5번째 줄의 MPI.Init에 의한 초기화와 9번째 줄의 MPI.Finalize에 의한 종료가 필수.
- 이 사이에 MPI의 API를 사용할 수 있게 됩니다.
- 6번째 줄의 MPI_Comm_size는 프로세스 수를 가져옵니다.
- 첫 번째 인수인 MPI_COMM_WORLD는 MPI 전체를 나타내는 변수이며, 두 번째 인수인 nproc에 프로세스 수를 부여하고 있습니다.
- 7번째 줄의 MPI_Comm_rank도 비슷한 기능으로, 각 프로세스가 담당하는 자신의 프로세스 번호를 얻을 수 있다. 이렇게 하면 모든 프로세스가 서로 다른 번호를 얻게 됩니다.
- MPI에서는 자신의 프로세스 번호를 '랭크'라고 부르니 기억해두자.

MPI 프로그램(데이터 교환)

test.c

```
#include "mpi.h"
main(int argc, char *argv[])
{
    int size, rank;
    double d[10], e[10];
    MPI_Status status;

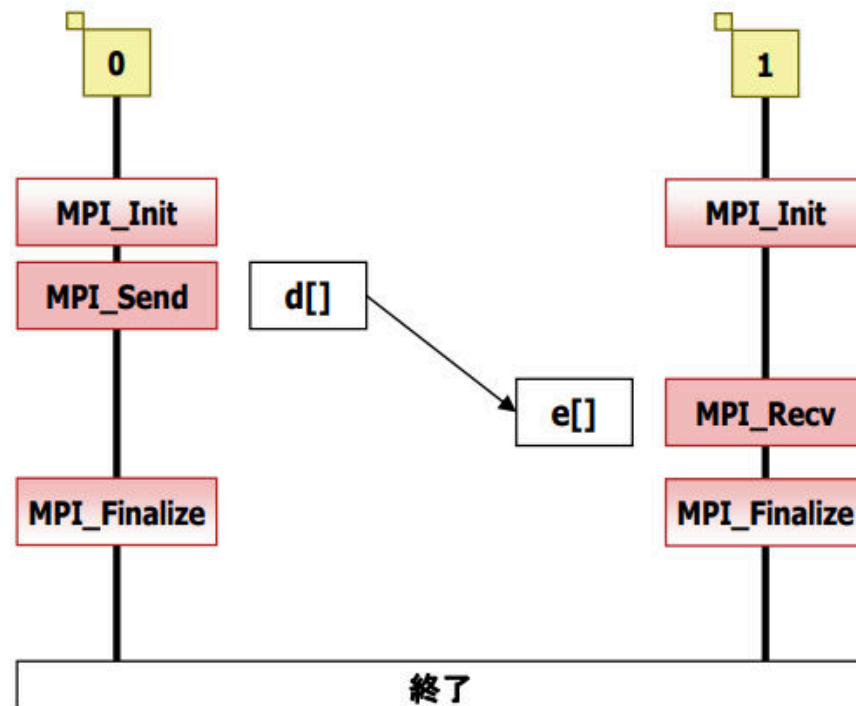
    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    switch(rank) {
    case(0):
        MPI_Send(d, 10, MPI_DOUBLE, 1,
        1000, MPI_COMM_WORLD); //to 1
        break;
    case(1):
        MPI_Recv(e, 10, MPI_DOUBLE, 0,
        1000, MPI_COMM_WORLD, &status); // from 0
        break;
    }

    MPI_Finalize();
}
```

```
$ mpicc test.c
$ mpiexec -n 2 -hostfile mpd.hosts ./a.out
```



MPI 프로그램(데이터 교환)

- 데이터 교환에서 중요한 것은 ① 송신(send), ② 수신(Recv), ③ 대기(Wait) 세 가지.
- MPI에서는 송신과 수신이 일치하지 않으면 통신이 되지 않음.
- 따라서 프로그램이 잘 흘러가도록 머리로 생각하면서 적절한 위치에 명령을 배치해야 함.
- 만약 실수로 인해 서로가 수신 대기 상태가 되면 처리가 전혀 진행되지 않는 '데드락(Deadlock)'이라는 현상이 발생하게 됨

● 전송(Send)

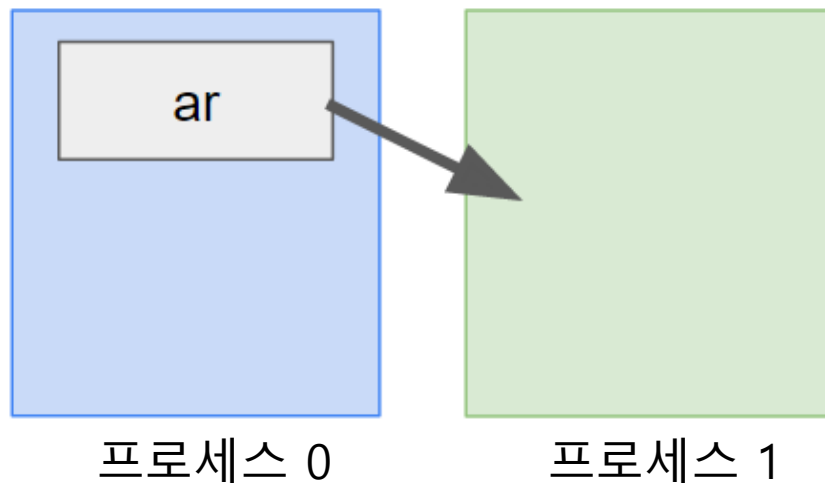
- 데이터를 다른 프로세스로 보내기 위한 API로 MPI_Send와 MPI_Isend가 있다.
- MPI_Send는 송수신이 확정될 때까지 대기하는 Blocking 통신이다.
 - ✓ 프로그램의 흐름에 따라 순서대로 실행되기 때문에 예상치 못한 일이 발생하기 어렵다는 특징이 있다.
- 반면 MPI_Isend는 송신 명령을 내리면서 다음 처리를 진행하는 Non-Blocking 통신이다.
 - ✓ 대기 시간이 없어 효율적이지만, 예상치 못한 데이터 업데이트가 발생하지 않도록 MPI_Wait를 통해 동기화 타이밍을 적절히 부여해야 한다.

MPI를 통한 병렬 컴퓨팅 프로그래밍

- 예를 들어 MPI_Send는 아래와 같이 작성할 수 있다.

```
int ar[10]; MPI_Send(ar, 10, MPI_INT, 1, 0, MPI_COMM_WORLD);
```

- MPI_Send의 인수는 왼쪽부터 순서대로 '보내고자 하는 배열의 시작 주소', '요소 수', '타입', '수신 측의 랭크 (프로세스 번호)', '태그(임의의 식별 번호)', 'MPI_COMM_WORLD'로 구성되어 있다.
- 이를 통해 랭크 1의 프로세스에 대해 배열 ar을 전송할 수 있다.



- MPI_Isend도 거의 동일한 인수 구성으로, 마지막에 Wait용 식별자를 붙이기만 하면 됨

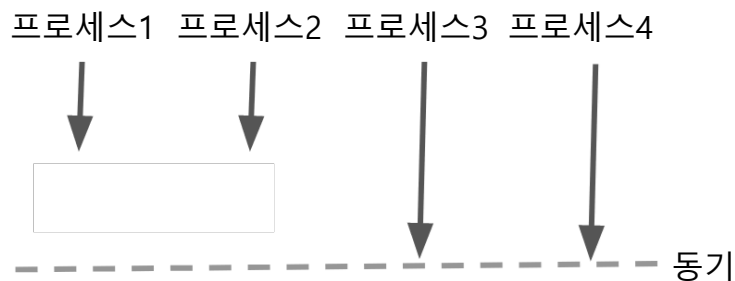
MPI를 통한 병렬 컴퓨팅 프로그래밍

- Non-Blocking MPI_Irecv의 경우 아래와 같다.

```
MPI_Request recv_request;  
int ar[10];  
MPI_Send(ar, 10, MPI_INT, 0, 0, MPI_COMM_WORLD, &recv_request);
```

● 대기(Wait)

- 블로킹 통신을 사용하든 비블로킹 통신을 사용하든, 노드 간 진행을 일치시키고 싶을 때가 있다.
- 예를 들어, 모든 데이터가 다 모인 후에 진행을 하고 싶을 때, 프로세스 3, 4가 따라잡을 때까지 프로세스 1, 2를 기다리게 하고 싶을 때가 있다.
- 이를 해결하기 위해 사용하는 것이 '동기화'입니다. 동기화를 통해 다른 프로세스를 기다릴 수 있다.



동기화

- 모든 노드를 동기화하려면 아래의 MPI_Barrier를 사용.
- MPI_Barrier(MPI_COMM_WORLD);
- 이를 송수신 뒤에 넣으면 노드들의 진행을 동기화할 수 있다.
- 비블로킹의 경우 Wait 또는 Waitall을 사용한다. 이들은 지정된 요청만 동기화할 수 있다.
- Waitall은 다음과 같이 사용함.

```
MPI_Request send_request, recv_request;  
. . .  
MPI_Isend( . . . , &send_request);  
MPI_Irecv( . . . , &recv_request);  
MPI_Waitall(1, &send_request, MPI_STATUS_IGNORE);  
MPI_Waitall(1, &recv_request, MPI_STATUS_IGNORE);
```

- 위 프로그램에서는 첫 번째 Waitall에서 모든 노드의 Isend를 기다리고, 두 번째 Waitall에서 모든 노드의 Irecv를 대기.
- MPI_Waitall의 인수는 왼쪽부터 순서대로 ' 요청 개수 ', ' 요청의 시작 주소 ', ' 상태'이다.
- 요청은 Isend나 Irecv의 끝에 붙이는 식별자를 의미하며, 이들과 대응한다.
- 상태는 기본적으로 MPI_STATUS_IGNORE를 사용하면 된다.
- 하지만 위의 경우 Waitall을 두 번 작성해야 하므로 번거롭습니다. 그래서 배열을 사용하여 Waitall을 단축해 보자.

동기화

```
MPI_Request request[2];  
. . .  
MPI_Isend( . . . , &(request[0]));  
MPI_Irecv( . . . , &(request[1]));  
MPI_Waitall(2, request, MPI_STATUS_IGNORE);
```

- 이렇게 배열을 Isend와 Irecv에 할당하면 한 번의 Waitall로 모든 통신을 회수할 수 있다.

간단한 MPI 프로그램 예시

1부터 100까지의 정수의 합을 2병렬로 구하는 프로그램

```
1  ✓ #include <stdio.h>
2  [ #include <mpi.h>
3  int main(argc, argv)
4  int argc;
5  char* argv[];
6  ✓ {
7  int i, istart, iend, isum, isum1;
8  int myrank, nprocs;
9  MPI_Status status;
10 MPI_Init(&argc, &argv); /* starts MPI */
11 MPI_Comm_size(MPI_COMM_WORLD, &nprocs); /* get number of processes */
12 MPI_Comm_rank(MPI_COMM_WORLD, &myrank); /* get current process ID */
13 istart = myrank * 50 + 1;
14 iend = (myrank + 1) * 50;      각 프로세스가 계산해야 할 범위 구하기
15 isum = 0;
16 for (i = istart; i <= iend; i++)  각 프로세스가 부분합을 계산
17     isum += i;
18 if (myrank == 1)
19     MPI_Send(&isum, 1, MPI_INT, 0, 100, MPI_COMM_WORLD);  프로세스 1은 부분합을 프로세스 0에 보냄
20 else
21     MPI_Recv(&isum1, 1, MPI_INT, 1, 100, MPI_COMM_WORLD, &status);  프로세스 0은 프로세스 1로부터 부분합을 수신함
22 if (myrank == 0)
23     printf("sum= %d\n", isum + isum1);  프로세스 0은 합계를 표시
24 MPI_Finalize();
25 return 0;
26 }
```

Parallel Programming with MPI

Parallel Programming with MPI

<https://github.com/csc-training/mpi-introduction?tab=readme-ov-file>

<https://www.open-mpi.org/doc/v4.0/>

MPI API(470개정도)

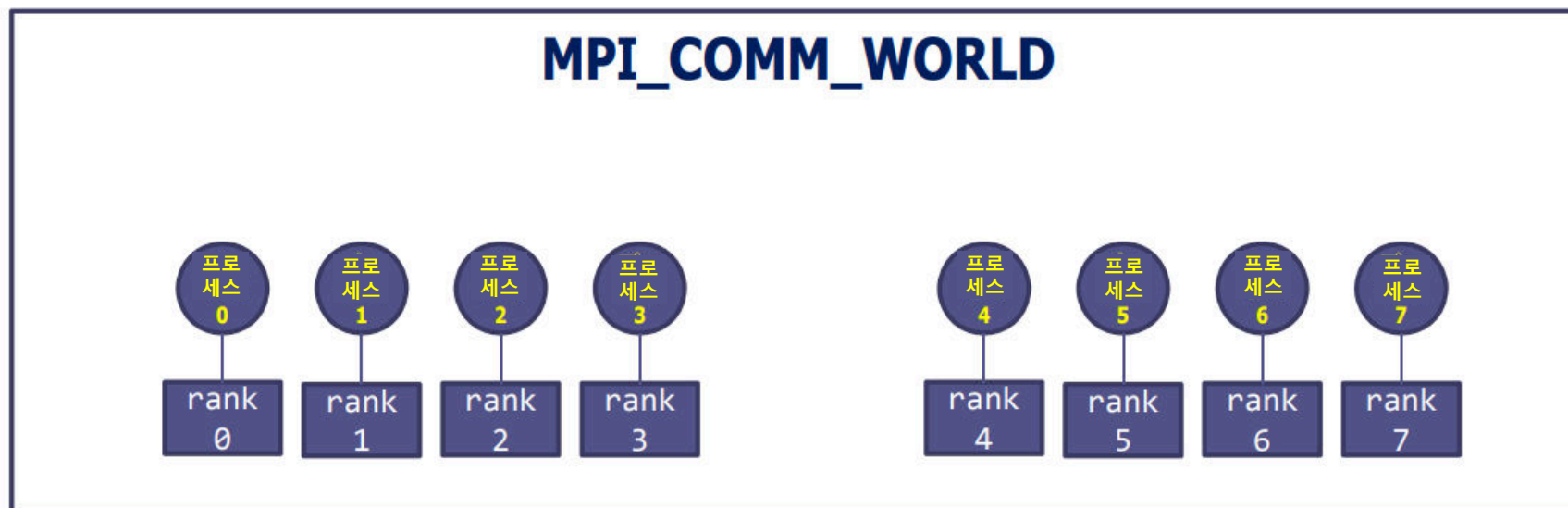
<https://docs.open-mpi.org/en/v5.0.x/man-openmpi/man3/index.html>

<https://github.com/open-mpi/ompi/wiki>

커뮤니케이터(Communicator)

- 커뮤니케이터(Communicator)
- 데이터 타입(Datatype)

- MPI에서 프로세스의 집단(집합)
- 집단적 조작 등에서의 조작 대상
 - 집단형 통신(Collective communication) : 통신이 커뮤니케이터 내에서 폐쇄적으로 이루어진다.
 - 집단형 I/O(Collective operation)
 - Window 조작(One-sided communication)
- MPI_COMM_WORLD: MPI 프로그램 시작 후 처음 생성되는 프로세스 전체 커뮤니케이터



커뮤니케이터(Communicator)

- 커뮤니케이터(Communicator)
 - 데이터 타입(Datatype)
-
- MPI에서 프로세스의 집단(집합)
 - 집단적 조작 등에서의 조작 대상
 - 집단형 통신(Collective communication) : 통신이 커뮤니케이터 내에서 폐쇄적으로 이루어진다.
 - 집단형 I/O(Collective operation)
 - Window 조작(One-sided communication)
 - 커뮤니케이터 분할: 새로운 랭크 번호가 할당된다.

